# Quick Draw Detection

CS 5480 – Project Final Report

**Colin Smith**
Computer Science
Missouri University of Science & Technology
Rolla, Missouri Phelps
cashpg@mst.edu

**Rohit Sayeeganesh**
Computer Science
Missouri University of Science & Technology
Rolla, Missouri Phelps
rsbby@mst.edu

**Aaron Muller**
Computer Science
Missouri University of Science & Technology
Rolla, Missouri Phelps
ajmdhf@mst.edu

**Rohith Velmurugan**
Computer Science
Missouri University of Science & Technology
Rolla, Missouri Phelps
rvgcp@mst.edu

## ABSTRACT

Sports climbing is inherently dangerous. When climbing, there is always the risk of falling. Sometimes this risk entails a potential ground fall. One critical component of safety when it comes to climbing is the quickdraw, which serves as an attachment point that connects the climbing rope to bolts installed on the rock face. Accurately identifying the position of quickdraws is essential for estimating fall risk and could determine whether a climber is protected from ground fall.

In this project we applied transfer learning to a YOLOv8n on our custom dataset of quickdraws. We built a custom dataset of 380 annotated sport climbing images sourced from the Missouri S&T Climbing Club's photo archive. We tested various techniques to determine which would give us the best results.

In this project we determined that utilizing feature extraction techniques provided the greatest improvement in results compared to our baseline resulting in a mAP50 of .43 compared to .2864. We then found that lowering our batch size from 16 to 8 also had a positive effect resulting in .458 mAP50.

## 1. INTRODUCTION

Sport climbing is an inherently dangerous sport. Several safety mechanisms have been created to reduce the risk. One way we can reduce the risk is by informing climbers if they are at risk of a ground fall. There are many factors that influence if a climber is at risk of a ground fall. An important one is the last point they attached to the wall at. In sport climbing, climbers use quickdraws to attach to the wall. These are their safety points, and their location must be identified to predict the risk of a ground fall.

Our goal is to use transfer learning to train a model to locate quickdraws in an outdoor setting. The ability to detect quickdraws will help climbing safety mechanisms to more accurately determine the risk of a ground fall.

There are many challenges when detecting quickdraws. At outdoor climbing locations, lighting and background vary vastly. The quickdraw could be in the shade on a dark rock, or it could be in the sun on a super light-colored rock. Another difficulty is the occlusion of quickdraws. Sometimes a climber may pass in front of part of the quickdraw; rope could cover a section, or the part of the wall could be covering part of the quickdraw. We must create a model that is capable of detecting the quickdraws in varying lighting conditions, varying backgrounds, and possible occlusion of the quickdraws.

## 2. RELATED WORK

We found a single paper published that related to detecting quickdraws. We used the YOLOv8n, a model that utilizes CIoU (Complete Intersection over Union) and DFL (Distribution Focal Loss) along with the transfer learning technique. Ivanova et al. [2020] used transfer learning to train a CNN to identify quickdraws while sport climbing and determine if there was a rope going through the quickdraw. However, in their study they detected the quickdraws on an indoor wall giving consistent lighting and background. They also tapped yellow boxes around their quickdraws to help locate it as locating was not their only goal, but they wanted to determine if rope was going through the quickdraw. Our sole goal is to locate the quickdraw.

In the paper, SCRDet: Towards More Robust Detection for Small, Cluttered and Rotated Objects, Yang et al. [2019] introduced techniques that could help the model identify quickdraws. Our project built on this slightly by augmenting images prior to object identification to regularize background and lighting, making identification easier. Buslaev et al. [2020] demonstrate several techniques for altering images that we used. This helped with optimizing images for object identification as well as increasing the size of our dataset.

## 3. METHODOLOGY

Our approach uses transfer learning on a YOLOv8n architecture pretrained on the COCO dataset. We fine-tuned this model on a custom dataset created from the Missouri S&T Climbing Club's photo archive. We evaluated several training configurations to identify which combination of techniques produce the best detection performance on our held-out test set.

### 3.1 Architecture

We selected the YOLOv8 nano variant as our base architecture. YOLOv8 is an object detector that performs bounding box predictions and classifications simultaneously. This made it well suited for real-time applications like detecting quickdraws. The

nano variant was chosen specifically because of our relatively small dataset and limited computational resources. The model is lightweight enough to be trained on CPU hardware without extreme runtimes while also achieving decent accuracy on detection tasks.

### 3.2 Training Setup

All models were trained using the default AdamW optimizer with a maximum of 100 epochs. Input images were 640x640 and had a batch size of 16 unless otherwise noted. Early stopping was applied with a patience of 10 epochs. This means that the training would halt if validation loss did not observe an improvement after 10 epochs.

### 3.3 Transfer Learning

For our feature extraction experiments, we initialized the YOLOv8n backbone with weights pretrained on the COCO dataset and froze the backbone layers during training. This meant that only the detection head, i.e. the layers responsible for predicting bounding boxes and class labels, were updated via gradient descent. We decided to go with this approach because we realized that the COCO-pretrained backbone had already learnt rich low-level and mid-level visual features like edges, textures, and shapes that transfer well to detecting objects like quickdraws. Freezing the backbone reduces the number of trainable parameters, which would then lower the risk of overfitting on our small dataset and reduce training time.

## 4. EXPERIMENTS

We ran four experimental configurations: a baseline, feature extraction with a frozen backbone and data augmentation, a model with updated image resolution, and a model with a reduced batch size.

### 4.1 Dataset

We created our own dataset from scratch. We got our images from the Missouri S&T Climb Club's photo archive. We annotated the images by hand labeling every quickdraw in the image. We ended up with 380 annotated images. The dataset consisted of a single quickdraw class. We split our data 70/15/15 into training, validation, and tests. This resulted in 266 training images, 57 validation images, and 57 test images.

### 4.2 Evaluation Metrics

To evaluate the models, we will compare their mean average precision (mAP) scores. mAP considers two main things. First, precision, a metric that describes what percent of the models' predictions were correct. Second, recall, which considers how many objects in the image we correctly identified. For an object to be considered correctly identified, we use intersection over union (IoU). This metric compares the ratio of the intersection area between the predicted and ground truth bounding boxes to their union area. We used these metrics to compare the 4 different models we trained.

### 4.3 Baseline

The first model we trained had no additional features and purely used the training/validation sets to predict quickdraws from the test set. We refer to this as the baseline, against which all subsequent models are compared.

The baseline was trained utilizing the YOLOv8 model, and we specifically utilized the nano variant because it was the most efficient for our dataset. This variant was used for every model, so we wanted to be consistent as to not skew results.

### 4.4 Feature Extraction

To leverage the pretrained abilities of this model, we broke our training into two stages. In the first stage, we froze the backbone for 12 epochs. This helped preserve the general object detection features learned in pretraining while allowing the head to adapt to quickdraw classification. In the second phase, we unfroze the backbone and continued training with early stopping enabled. This allowed the full network to now fine-tune jointly. This phase ended up running about 40 epochs.

### 4.5 Larger Image Size

Since quickdraws can occupy a small portion of the image, we wondered if increasing input resolution would be able to improve the model's abilities to detect the quickdraws. In this experiment, we updated the model's image size from 640x640 to 768x768. We also utilized feature extraction in this model for the benefits stated earlier. Training restarted from the base model but still employs the feature extraction technique discussed earlier.

### 4.6 Reduced Batch Size

The last technique we utilized was reducing our batch size. Every other model we trained had a batch size of 16. For testing this technique, we used a batch size of 8. Smaller batch sizes can introduce more stochasticity into the gradient updates. This can act as an implicit regularizer and help reduce the risk of overfitting. We also utilized feature extraction in this model for the benefits stated earlier. Training restarted from the base model but still employs the feature extraction technique discussed earlier.

## 5. RESULTS

Table 1 displays the model's performance on our set of test data.

**Table 1. Model Test Results**

| Model | mAP50 | mAP50-95 | Precision | Recall |
|---|---|---|---|---|
| Baseline | 0.2864 | 0.0901 | 0.3834 | 0.3536 |
| Feature Extraction | .43 | .163 | .563 | .425 |
| Larger Image Size | .432 | .152 | .555 | .448 |
| Reduced Batch Size | .458 | .177 | .642 | .453 |

### 5.1 Baseline

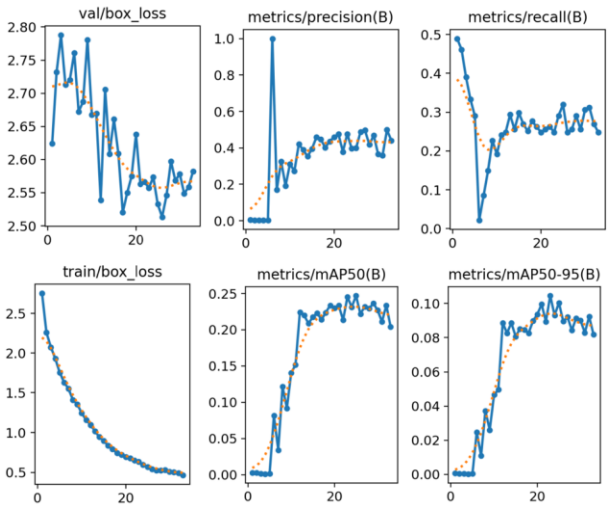


**Figure 1: Baseline metrics.**

In figure 1 we see the training loss steadily decrease throughout all of the training. The validation loss starts off by decreasing by starting to increase at the end. Both mAP50 and mAP50-95 increase at first but plateau and start to decrease towards the end. Precision and recall both start off with big jumps, then plateau for the majority of training.

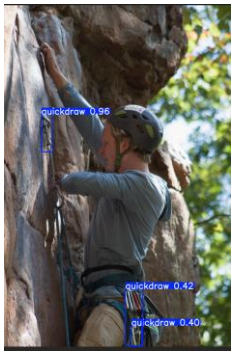


**Figure 2: Baseline model prediction on test image.**

Figure 2 is the output from the baseline model. It shows that 3 quickdraws were detected in the image. It looks like the model accurately identified the quickdraws on the climber's harness; however, it only identified half of the quickdraw on the wall.

## 5.2 Feature Extraction

After employing feature extraction, the model had a 50.14% increase in its mAP50 score and an 80.91% increase in its map50-95 score. Precision and recall also improved by 46.84% and 20.19% respectively.

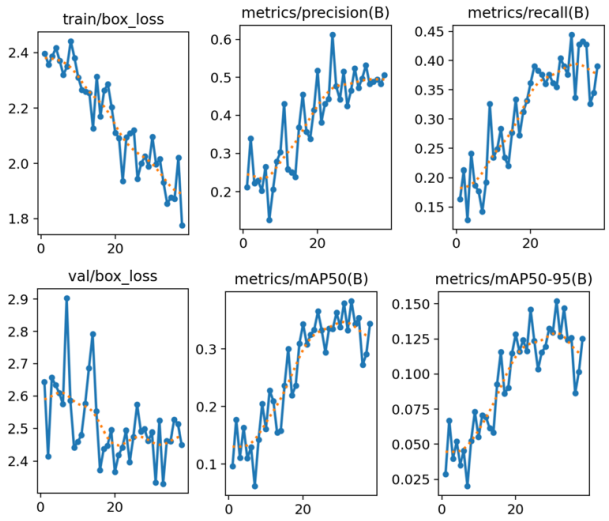


**Figure 3: Feature extraction metrics.**

In figure 3 we can see the training loss generally decrease over the epochs. Validation loss decreases until epoch 20 where it just starts to oscillate. The mAP50 and mAP50-95 scores increased throughout most of the training but slightly decreased at the end.

## 5.3 Larger Image Size

Increasing the input image size resulted in a 50.84% increase in the mAP50 score and a 68.70% increase in mAP50-95 score when compared to the baseline. When compared to the feature extraction model, they had similar mAP50 scores; however, the mAP50-95 was 7.24% more accurate than the model with the larger input image size. The base feature extraction model also had a 1.44% higher precision score, but the model with the larger input size had a 5.41% higher recall score.

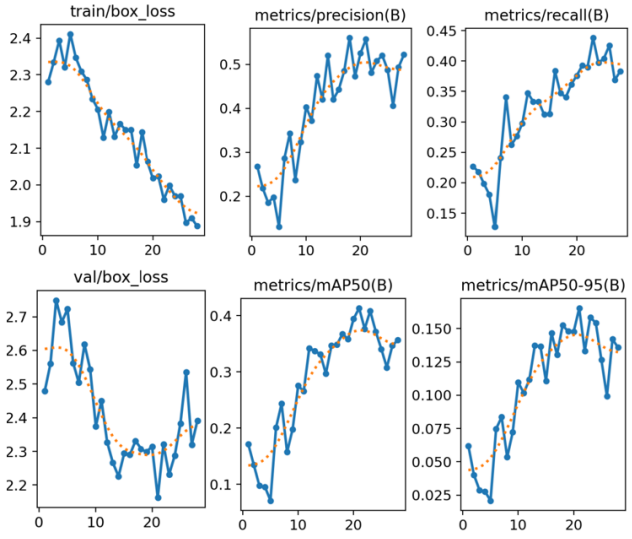


**Figure 4: Increased image size metrics.**

As seen in figure 4, the training loss saw a steady decrease while the validation loss decreased at first and started increasing towards the end. Similarly, both mAP50 and mAP50-95 increased at first but then finished by slightly decreasing.

### 5.4 Reduced Batch Size

Reducing the models' training batch size resulted in a 59.92% increase in its mAP50 score from the baseline and a 6.51% increase compared to the base feature extraction model. It also showed a 8.59% increase in the mAP50-95 score relative to the feature extraction model.

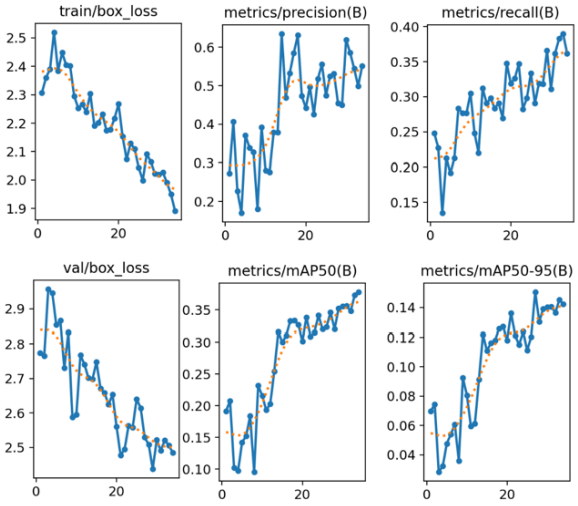


**Figure 5: Lower batch size metrics.**

In figure 5, we can see a steady decrease from both training and validation of box loss. We also see a steady increase from both the mAP50 and mAP50-95 score. The recall score also generally increases. Precision increases at first but then seems to oscillate around .5.

## 6. ANALYSIS

### 6.1 What Worked

From our models, the most effective configuration was the model that had a lower batch size and utilized feature extraction. It achieved a mAP50 of 0.458 compared to the baseline's 0.2864. This suggests that freezing the pretrained backbone and reducing the batch size preserved generalizable low-level features and helped reduce overfitting.

Feature extraction seemed to be the most valuable resource while cross training the YOLOv8n model. The model where we only employed feature extraction increased the mAP50 score by .1436, or 50.14%. Similarly, the model where we increased input image size and the mode where we decreased batch size also used feature extraction, and they also outperformed the baseline model even more when considering mAP50. This shows that using feature extraction techniques to preserve the backbone of the model directly improves the model overall.

Reducing batch size also had a positive effect. In our model that we reduced the batch size to 8 and utilized feature extraction techniques, we saw very good results. It achieved a mAP50 of .458. This is .1716, or 59.92% higher than the baseline. It is also .028 higher than the model, which only had feature extraction. This model also is the only model that didn't show signs of overfitting. The training loss and validation loss both decrease the entire time. Compared to the other models where training loss decreased while validation loss would plateau or start to rise again. This is likely due to the stochasticity of the lower batch size. Decreasing the batch size makes the averages per batch a little more random, thus acting somewhat as a regularizer since it makes it harder for the neurons to just memorize the training data patterns.

Overall, reducing the batch size combined with feature extraction resulted in a model that had the highest metrics in every category and didn't show signs of severe overfitting, telling us that feature extraction combined with the lower batch size produced the best results given our dataset.

### 6.2 What Didn't Work

One method that didn't work was increasing the input image size. The mAP50 score stayed like the model where we only implemented feature extraction. We saw the recall rise, but the precision fell resulting in the mAP50 balancing out. By increasing the input resolution, we increased the amount of data passed to our model, so the quickdraws that were smaller in the image had a higher chance of being seen. This is why we believe that recall has increased. The higher resolution lets more quickdraws get detected. However, the extra noise that also got passed onto the model impacted it negatively. It made the model more likely to predict something that is not a quickdraw to be a quickdraw.

Another issue we ran into was overfitting. Every model, except for the model with a reduced batch size, saw a divergence between training loss and validation loss. This indicates that the models were all overfitting and just memorizing the training data. Given that our data set contained about 380 images in total, we had only 266 images for our training set. If we had a larger dataset, it may have allowed the baseline to train for longer, allowing the model to converge rather than be forced to stop early.

### 6.3 Comparison Summary

Compared to baseline, here's how each model did:

**Table 2. Models compared to the baseline**

| Model | ΔmAP50 | ΔmAP50-95 | ΔPrecision | ΔRecall |
|---|---|---|---|---|
| Feature Extraction | 0.1436 | 0.0729 | 0.1796 | 0.0714 |
| Larger Image Size | 0.1456 | 0.0619 | 0.1716 | 0.0944 |
| Reduced Batch Size | 0.1716 | 0.0869 | 0.2586 | 0.0994 |

In table 2, every metric yielded a positive delta across all three configurations which used feature extraction. This reinforces that feature extraction was a key driver of improvement over the baseline.

**Table 3. Models that used feature extraction compared to feature extraction baseline**

| Model | ΔmAP50 | ΔmAP50-95 | ΔPrecision | ΔRecall |
|---|---|---|---|---|
| Larger Image Size | 0.001 | -0.011 | -0.008 | 0.023 |
| Reduced Batch Size | 0.028 | 0.014 | 0.079 | 0.028 |

In table 3, we compare the models which use feature extraction. This table makes it obvious which model is superior. As discussed earlier, increasing the input image size allowed for better recall, but lowered precision, thus making the change cancel itself out.

# 7. CONCLUSION

In this project we applied transfer learning using a YOLOv8n architecture to the problem of quickdraw detection in outdoor sports climbing environments. We evaluated four different configurations. Through this, we found that the model that had reduced batch size of 8 and utilized feature extraction techniques produced the best overall results with a mAP50 of 0.458.

These results demonstrate that lightweight and transfer-learned object detectors are a viable option for climbing safety systems, and with this in mind, accurate quickdraw classification is an achievable goal to ultimately lower the inherent danger of sports climbing.

Our greatest limitation was our dataset. While we got encouraging results, 380 images are not enough for us to train our models till convergence. Another limitation was choosing the nano variant of the YOLOv8 models. This was chosen with regards to how it would be deployed in industry, outside on someone's phone. However, choosing a larger model would give us better representational power, but to effectively use that power, we would need a larger dataset.

If we continued the project, the next thing we would do is expand our dataset. One of the main issues we faced was overfitting. Having a larger dataset would do more to solve this than any hyperparameter adjustments, so so solving this issue would be our priority. Second, we would experiment solely with batch size. Compare batch sizes of 4, 8, and 16 explicitly. Then we would also like to compare other forms of regularization to the model where we lowered batch size.

# 8. APPENDIX

GitHub Repository: https://github.com/RohitCodes22/Quickdraw-Detection---CS-5480-Final-Project

All experiments were run using Python 3.12.1 with Ultralytics YOLOv8 version 8.4.41 and PyTorch 2.11.0. Training was performed on an HP Spectre with an Intel Core i7 Evo processor and an HP Envy x360 with an Intel Core i7 processor. The dataset, annotation files, and training configuration files are available in the linked GitHub repository above.

## 8.1 Baseline Extra Figures

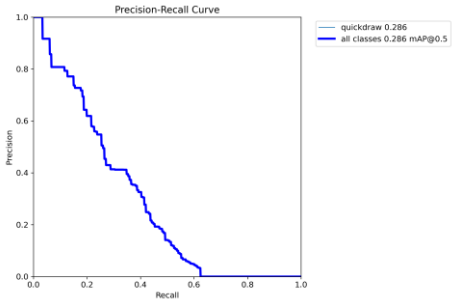


**Figure 6: Precision-Recall curve for baseline.**

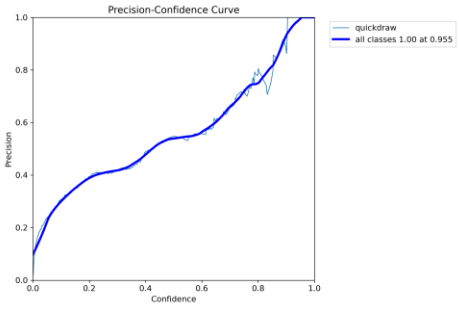


**Figure 7: Precision-Confidence curve for baseline.**

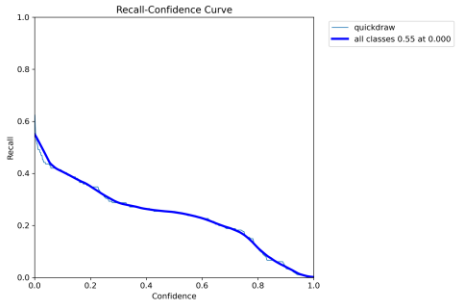


**Figure 8: Recall-Confidence curve for baseline.**

## 8.2 Feature Extraction Extra Figures

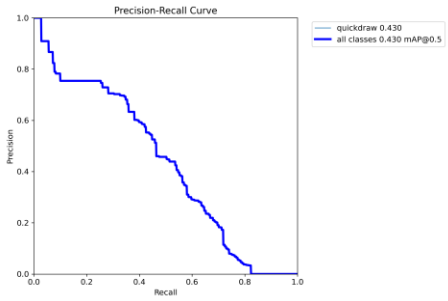


**Figure 9: Precision-Recall curve for feature extraction.**

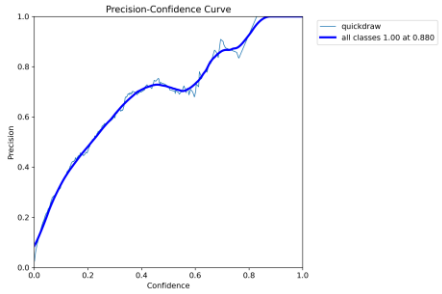


**Figure 10: Precision-Confidence curve for feature extraction.**

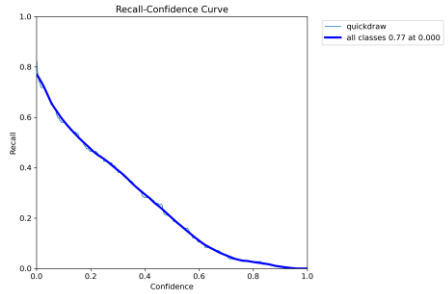


**Figure 11: Recall-Confidence curve for feature extraction.**

## 8.3 Larger Input Image Size Extra Figures

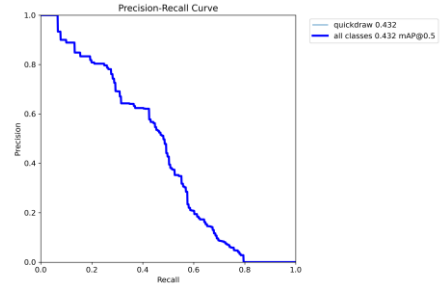


**Figure 12: Precision-Recall curve for larger input image size.**

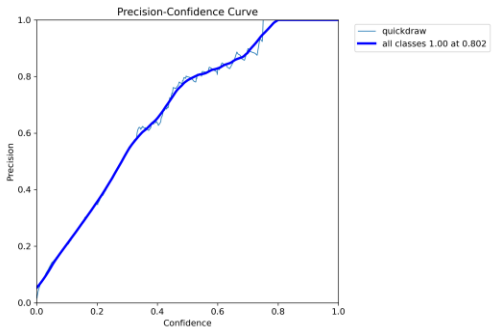


**Figure 13: Precision-Confidence curve for larger input image size.**

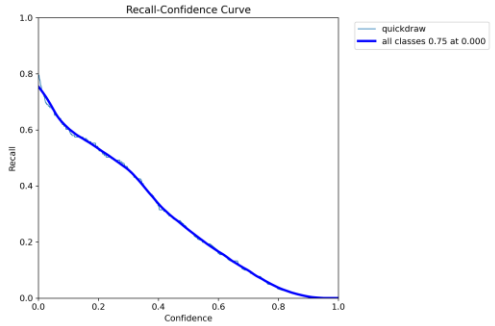


**Figure 14: Recall-Confidence curve for larger input image size.**

## 8.4 Reduced Batch Size Extra Figures

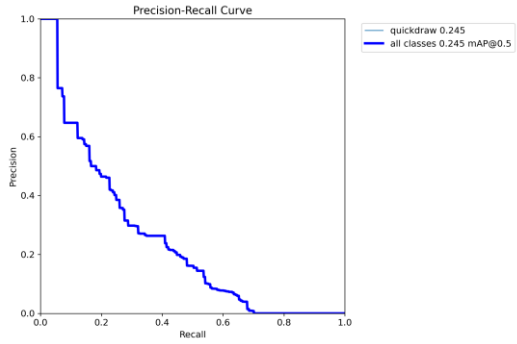


**Figure 15: Precision-Recall curve for reduced batch size.**

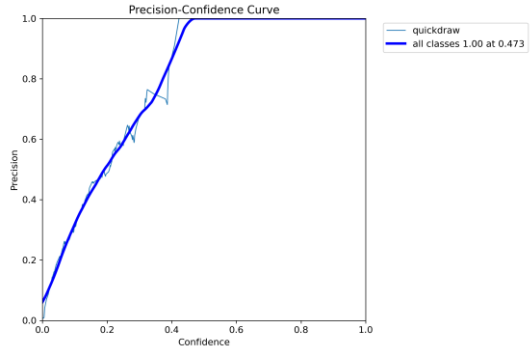


**Figure 16: Precision-Confidence curve for reduced batch size.**

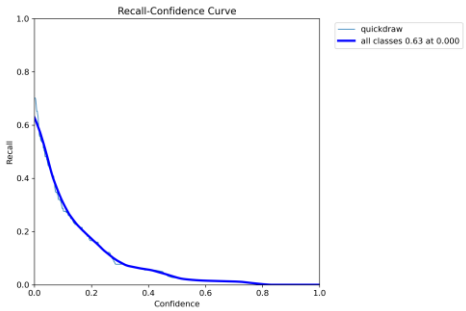


**Figure 17: Recall-Confidence curve for reduced batch size.**

## REFERENCES

[1] Iustina Ivanova, Marina Andrić, Sadaf Moaveninejad, Andrea Janes, and Francesco Ricci. 2020. Video and Sensor-Based Rope Pulling Detection in Sport Climbing. In Proceedings of the 3rd International Workshop on Multimedia Content Analysis in Sports (MMSports '20). Association for Computing Machinery, New York, NY, USA, 53–60. https://doi.org/10.1145/3422844.3423058

[2] Xue Yang, Jirui Yang, Junchi Yan, Yue Zhang, Tengfei Zhang, Zhi Guo, Sun Xian, and Kun Fu. 2019. SCRDet: Towards More Robust Detection for Small, Cluttered and Rotated Objects. https://arxiv.org/abs/1811.07126

[3] Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M., & Kalinin, A. A. (2020). Albumentations: Fast and Flexible Image Augmentations. Information, 11(2), 125. https://doi.org/10.3390/info11020125